

06 — Modeling Roadmap

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	MR-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Research Design , Compute Memo , Roadmap

Overview

This roadmap defines a four-phase progression from literature review through pretrained-model exploration. Each phase builds on the previous one and includes explicit **decision gates** — criteria that must be met before progressing to the next phase.



Phase 0: Literature and Data Curation

Objective

Build the foundational knowledge and data infrastructure required for all subsequent modeling work.

Activities

Activity	Output	Duration
Literature survey of sequence-based regulatory prediction	Annotated bibliography (≥ 20 papers)	2 weeks
Literature survey of interpretable ML in genomics	Summary of interpretability methods applicable to genomics	1 week
Download and curate ENCODE cCREs for K562 and GM12878	Versioned BED files in data/raw/	3 days

Download hg38 reference genome (per-chromosome FASTA)	Reference files in data/reference/	1 day
Download JASPAR motif database	PFM files in data/raw/	1 day
Download GENCODE annotations	GTF file for coding region exclusion	1 day
Implement data pipeline: parse BED → extract sequences → generate negatives → split	src/data/ modules; cached features in data/processed/	1 week
Exploratory data analysis	Notebook with sequence length distributions, GC content, element-type breakdown, chromosomal distribution	3 days

Deliverables

- ☐ Annotated bibliography
- ☐ Curated dataset uploaded to Kaggle (private dataset)
- ☐ Functional data pipeline (sequence extraction, negative sampling, chromosome-holdout split)
- ☐ EDA notebook with visualizations
- ☐ k-mer feature computation verified

Decision Gate 0→1

Criterion	Required?
≥ 50K positive regions extracted and verified	Yes
≥ 50K negative regions generated, GC-matched, non-overlapping	Yes
Chromosome-holdout split implemented and validated	Yes
At least k-mer (k=4) features computed for all regions	Yes
No obvious data leakage identified in EDA	Yes

Phase 1: Classical Interpretable Baseline

Objective

Train and evaluate interpretable classical ML models that serve as the project's primary baseline and first research contribution.

Models

Model	Features	Interpretability mechanism	Kaggle feasible?
Logistic Regression (L1/L2)	k-mer frequencies (k=4,5,6)	Coefficient magnitudes; per-feature weights	✓ (CPU, minutes)

Random Forest (500–1000 trees)	k-mer + GC + dinucleotide	Gini importance; permutation importance	✓ (CPU, 5–20 min)
XGBoost / LightGBM	k-mer + GC + dinucleotide + motif scores	SHAP values; feature importance	✓ (CPU, 5–20 min)

Experiment Plan

Experiment	Description	Metrics
E1: Baseline classification	Train LogReg, RF, XGB on k=4 k-mer features	AUROC, AUPRC, F1
E3: k-mer resolution sweep	Compare k=3, 4, 5, 6 on RF	AUROC per k value
E2: Feature ablation	k-mer only → +GC → +dinuc → +motif on RF	AUROC delta per feature set
E7: Negative set sensitivity	Compare random vs. GC-matched vs. flanking negatives	AUROC variation
Trivial baselines	GC-only LogReg; random classifier; length-only classifier	Expected AUROC ~0.50–0.65

Interpretability Activities

Activity	Output
Extract top-20 k-mer features from RF	Ranked feature list
Compute SHAP summary plot for XGBoost	SHAP plot (PNG/SVG)
Cross-reference top features with JASPAR motifs	Motif overlap table
Compute per-feature permutation importance	Importance ranking with confidence intervals

Deliverables

- ☐ Trained LogReg, RF, XGBoost models with logged metrics
- ☐ Model comparison table
- ☐ Feature importance analysis per model
- ☐ SHAP analysis for XGBoost
- ☐ Motif cross-reference report
- ☐ Negative set sensitivity analysis
- ☐ Reproducible Kaggle notebook

Decision Gate 1→2

Criterion	Required?
At least one model achieves AUROC ≥ 0.75 on test set	Yes

Feature importance output is generated and biologically plausible	Yes
Trivial baselines are convincingly outperformed	Yes
Negative set sensitivity analysis completed (AUROC stable across strategies)	Yes
All experiments are logged and reproducible	Yes
Proceed only if team wants to explore DL (no mandatory advancement)	Recommended

Phase 2: Lightweight Deep Learning Baseline

Objective

Train shallow CNNs on one-hot encoded sequences to learn sequence features automatically, and compare their predictions and interpretability to classical baselines.

Models

Model	Architecture	Parameters	Input	Kaggle feasible?
Shallow CNN	2 conv layers (64, 128 filters), global max pool, 2 FC layers	~200K–500K	One-hot (1000 × 4)	✓ (GPU, 30–90 min)
Deeper CNN	3–4 conv layers with batch norm	~500K–2M	One-hot (1000 × 4)	✓ (GPU, 1–3 hr)
CNN + Attention	CNN encoder + single attention layer	~500K–2M	One-hot (1000 × 4)	✓ (GPU, 1–3 hr)

Architecture: Shallow CNN (Reference Design)

```
inputs = (batch, 1000, 4) # One-hot encoded 1 kb sequence
conv1d(4 → 64, kernel=15, ReLU) # Learn 15-mer-scale patterns
maxPool1d(pool=4) # Downsample to 250 positions
conv1d(64 → 128, kernel=7, ReLU) # Learn higher-order patterns
GlobalMaxPool1d() # 128-dimensional representation
linear(128 → 64, ReLU, Dropout) # Classifier head
linear(64 → 1, Sigmoid) # Binary output
```

Interpretability Mechanisms

Method	Implementation	Output
Filter visualization	Extract and visualize first-layer convolutional filters as PWMs	Sequence logo per filter
Saliency maps	Gradient × input for per-nucleotide attribution	Heatmap per sequence
Integrated Gradients	<code>captum.attr.IntegratedGradients</code>	Per-nucleotide importance scores
Filter-to-motif matching	Convert learned filters to PWMs → scan against JASPAR	Filter-motif similarity table
Attention weights (if attention used)	Visualize attention distribution along sequence	Attention heatmap (with caveats about interpretability of attention)

Experiment Plan

Experiment	Description	Metrics
E6a: Shallow CNN	Train shallow CNN; compare AUROC to classical baselines	AUROC, AUPRC, F1, train time
E6b: Filter analysis	Extract and visualize CNN filters; compare to JASPAR	Filter-motif overlap %
E6c: Saliency analysis	Generate saliency maps for test-set predictions	Qualitative inspection + motif overlap
E6d: CNN vs. classical	Side-by-side: accuracy, interpretability, compute cost	Comparison matrix

Deliverables

- ☐ Trained CNN model with logged metrics
- ☐ CNN filter visualization and motif comparison
- ☐ Saliency maps / integrated gradients for selected test regions
- ☐ Accuracy vs. interpretability trade-off analysis
- ☐ Updated model comparison table (classical + CNN)
- ☐ Reproducible Kaggle notebook

Decision Gate 2→3

Criterion	Required?
CNN achieves AUROC ≥ 0.80 on test set	Yes
At least 3 CNN filters match known TF motifs	Yes
Saliency-based interpretability produces biologically plausible outputs	Yes

Trade-off analysis between CNN and classical models is documented	Yes
Team assesses that pretrained embeddings could add value	Recommended
Pretrained model can be loaded on Kaggle without exceeding memory	Required for Phase 3

Phase 3: Pretrained Genomic Embedding Experiments

Objective

Explore whether pretrained genomic model embeddings improve regulatory classification, and compare interpretability between pretrained-embedding pipelines and the project's interpretable baselines.

Critical note: This phase involves using pretrained models for inference/embedding extraction only. Training a genomic foundation model from scratch is out of scope. This is a later-stage exploration, not a day-one requirement.

Candidate Pretrained Models

Model	Parameters	Max input length	Available on Kaggle?	Feasibility
DNABERT-2	~117M	512 tokens ($\approx$ ~1.5 kb with BPE)	Via HuggingFace; manually upload	Inference feasible on Kaggle GPU
Nucleotide Transformer (smallest)	~50M–500M	1000–6000 bp (varies by version)	Via HuggingFace	Smallest version feasible
HyenaDNA (small)	~1.6M–6.6M	Up to 1 Mb (small version: shorter)	Via HuggingFace	Feasible (small version)
AlphaGenome	Large	Very long context	Available on Kaggle Models	Inference only; memory-constrained

Approach

Pretrained Model (frozen or partially fine-tuned)

Extract embeddings for each 1 kb region

$\Rightarrow (N, \text{embedding_dim})$ matrix

Option A: Embedding + simple classifier (LogReg, RF, xGB)

$\Rightarrow$ Maintains some interpretability via SHAP on classifier

Option B: Fine-tune last few layers + classifier head

$\Rightarrow$ Higher accuracy potential; reduced interpretability

Option C: Concatenate embedding + k-mer features $\Rightarrow$ classifier

$\Rightarrow$ Hybrid: combines learned and engineered features

Experiment Plan

Experiment	Description	Metrics
------------	-------------	---------

E8a: Embedding extraction	Extract embeddings from smallest feasible pretrained model	Embedding quality (AUROC of simple classifier)
E8b: Embedding classifier	Train LogReg/RF on pretrained embeddings	AUROC vs. k-mer baseline
E8c: Hybrid features	k-mer + pretrained embedding → XGBoost	AUROC improvement over either alone
E8d: Interpretability comparison	SHAP on hybrid vs. k-mer-only	Feature attribution quality
E8e: Limited fine-tuning	Fine-tune last 2 layers + classifier (if memory allows)	AUROC; training time

Deliverables

- ☐ Embeddings extracted and cached
- ☐ Embedding-based classifier trained and evaluated
- ☐ Comparison: pretrained vs. classical vs. CNN
- ☐ Interpretability analysis on pretrained pipeline
- ☐ Documented trade-offs (accuracy, interpretability, compute cost)

Decision Gate 3→4

Criterion	Required?
Pretrained embeddings provide ≥ 0.02 AUROC improvement over best baseline	Recommended (not blocking)
Interpretability comparison is documented	Yes
Compute cost is documented and sustainable	Yes
Team identifies concrete multi-omics or cell-type-specific extensions worth pursuing	Required for Phase 4

Phase 4: Extensibility into Multi-Omics or Cell-Type-Specific Prediction

Objective

Extend the system beyond sequence-only, single-cell-type prediction to incorporate additional data modalities and broader biological questions.

Note: This phase is aspirational. It requires a stable baseline from Phases 1–3 and potentially additional data access (e.g., GTEx). It is documented for continuity, not as a commitment.

Candidate Extensions

Extension	What it adds	Data source	Complexity
-----------	--------------	-------------	------------

Multi-cell-type regulatory prediction	Predict cell-type-specific regulatory activity from sequence	ENCODE cCREs across 5+ cell types	Moderate
Expression-linked regulatory scoring	Link regulatory predictions to nearby gene expression	GTEx summary data	Moderate-High
Cross-species regulatory conservation	Compare regulatory predictions between human (hg38) and mouse (mm10)	ENCODE mouse + mm10	High
Variant effect prediction	Predict how SNPs in non-coding regions alter regulatory predictions	dbSNP / ClinVar	High
Single-cell accessibility	Cell-type-specific accessibility at single-cell resolution	Published scATAC-seq datasets	High

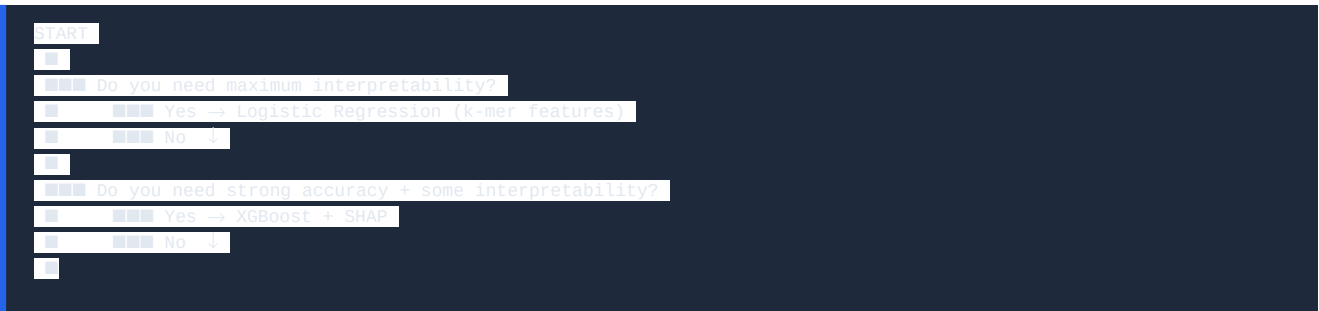
Phase 4 is NOT a Commitment

This phase exists to document where the project could go. It does not imply that the team will reach this phase. The project is successful if it completes Phases 1–2 with strong results.

Decision Gate Summary Table

Gate	From → To	Key criterion	Hard requirement?
0 → 1	Data → Classical ML	≥ 50K positive/negative regions; pipeline functional	Yes
1 → 2	Classical → CNN	AUROC ≥ 0.75 with classical model; interpretability verified	Yes
2 → 3	CNN → Pretrained	AUROC ≥ 0.80 with CNN; filters match motifs; team wants to proceed	Partially (AUROC is hard; team desire is soft)
3 → 4	Pretrained → Multi-omics	Stable baseline; concrete extension identified; data available	No (aspirational)

Model Selection Decision Tree




```
■■■ Do you want to discover motif-like patterns from data?
■    ■■■ Yes → Shallow CNN + filter visualization + saliency
■    ■■■ No  ↓
■
■■■ Do you want state-of-the-art sequence representation?
■    ■■■ Yes, and Kaggle memory allows → Pretrained embedding + simple classifier
■    ■■■ No, or memory doesn't allow → Stay with CNN or XGBoost
■
■■■ Default recommendation: XGBoost + SHAP (best balance for this project)
```

End of Document 06 — Modeling Roadmap
Next: [07 — Compute & Feasibility Memo](#)